TATA
TATA ELXSI

TELIPORT
Script Your Future
Season 4

Case Study No. 3

Adaptive QoS Engine for Mixed Home Broadband Traffic

Team Details

Team Name: **DataStructure**

Team Members: Rahul Atkare | Akash Bhuyan | Rushikesh Kedar

THINK. BUILD. CREATE. WIN

TELIPORT
Script Your Future
Season 4

QoSense — Adaptive, Intent-Aware Home QoS Engine

Protect calls and games. Keep downloads moving. Adapt to changing network conditions without reading private data.

DataStructure

TATA
TATA ELXSI

The problem: one home connection carries calls, games, video and big downloads at once. A big download fills the router's waiting line, so calls and games lag. Fixed rules cannot tell what matters right now.

THE PROBLEM

Lag comes from waiting, not slow internet



The call packet waits behind the whole queue, so delay and jitter jump. This is called **bufferbloat**.

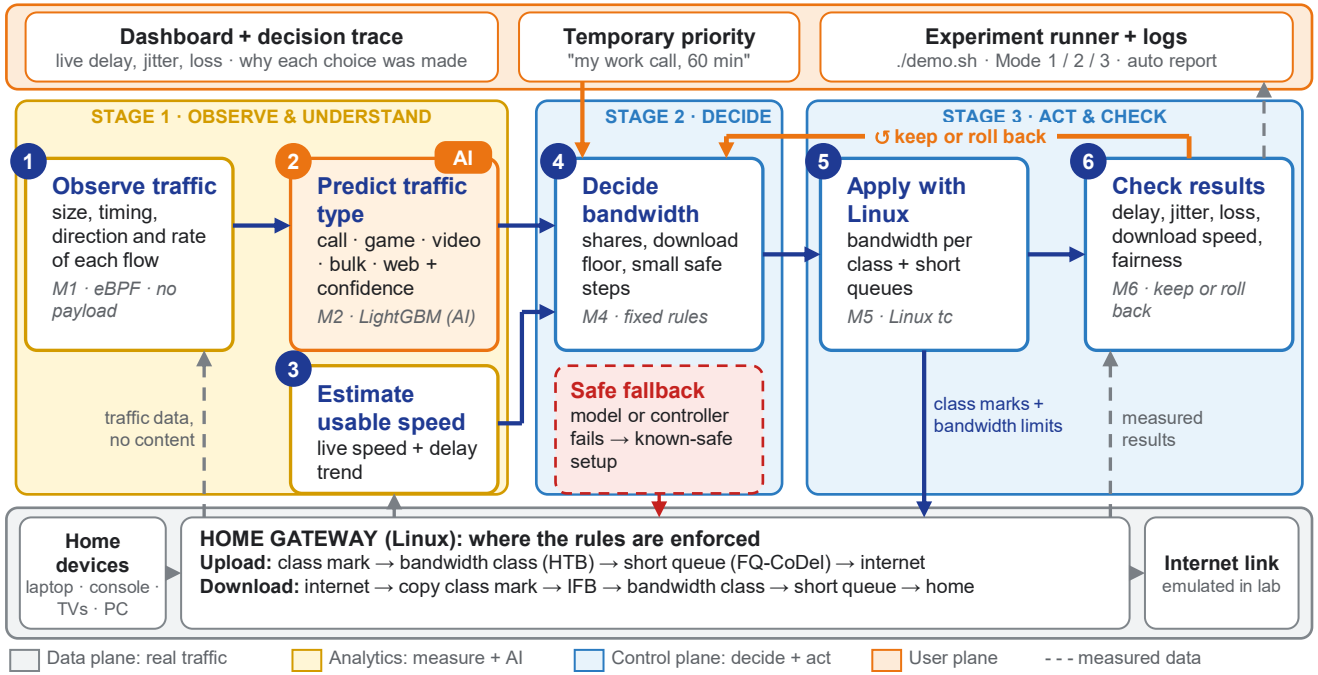
1. **Queues, not speed, cause lag.** Big downloads fill oversized buffers.
2. **Fixed rules break.** Traffic is encrypted, ports change, needs change.
3. **Strict priority is unfair.** Downloads and updates can stop completely.

The question we will test

Can learning how traffic behaves give better bandwidth decisions than fixed rules, even in network conditions the system has never seen?

We answer it with a fair experiment. We do not assume the answer.

HOW IT WORKS: OBSERVE → PREDICT → DECIDE → APPLY → CHECK



THE AI, IN ONE LOOK

Model: LightGBM, fast decision trees for numeric data

It sees: packet sizes, timing, direction, rate and duration of each flow. Never the content.

It predicts: call · game · video · bulk · web, plus how confident it is.

How it helps: the predicted type decides which bandwidth share the flow gets.

If unsure: safe best-effort class. The case is logged and can be corrected.

Why LightGBM: small and fast on flow numbers, shows which features mattered, gives confidence. Deep models need far more data and compute.

Its limit: AI only suggests a type. Fixed, checked rules decide and apply every change.

Safety built in

- Calls protected; downloads always keep a minimum share
- Temporary priority expires on its own
- Small, limited changes; no flip-flopping
- Every change measured; worse → roll back

WHY QOSENSE IS DIFFERENT

Common approach	Where it falls short	What QoSense adds
Fixed priority	does not adapt; low priority can starve	**priorities follow live traffic + minimum shares**
Port / DSCP rules	ports and marks often hide the real app	**learns how the traffic behaves**
Deep packet inspection	reads content; blocked by encryption	**uses traffic data only, no decryption**
Fair queues (FQ-CoDel)	short queues, but no app shares or user intent	**builds on them + class shares + intent**
Static shaping	wrong when internet speed changes	**live usable-speed estimate**
Black-box AI control	hard to limit or explain	**AI only classifies; rules act; logs explain**
No feedback	cannot tell if a change helped	**measure, then keep or roll back**

PROOF: SAME TESTS, THREE MODES

Design stage: all values are targets, not results.

MODE 1 · No QoS — reference: no special handling

MODE 2 · Rule-based — same pipeline, fixed rules classify

MODE 3 · QoSense AI — same pipeline, LightGBM classifies

Only the classifier differs between Mode 2 and Mode 3, so any gain can be traced to the AI.

TARGETS — TO BE VALIDATED

- **≤ 30 ms** extra call delay under load
- **≤ 10 ms** jitter (delay variation)
- **< 1%** lost call packets
- **0 s** download stalled
- **≥ 0.9** fair share across 3 TVs
- **≤ 5 s** re-adapt after speed drop
- **AI > rules** classifier on unseen runs
- **measured** CPU + AI decision time

10 runs per mode · same seeds · held-out runs with new speeds, delays and random ports · median + 95% range

TEST SCENARIOS (EMULATED IN THE LAB)

1 work call + big download **2** speed drops 100 → 20 Mbps **3** 3 TVs + online gaming
One command runs each: **./demo.sh iso_during_call**

ROUND 1 (PLANNED) → ROUND 2

R1: working lab, rule baseline, LightGBM, real Linux enforcement, before/after report, dashboard
R2: more traffic types, confidence calibration, harder speed changes, more runs, router hardware

PROTOTYPE → PRODUCTION

Now: Ubuntu VM, isolated lab, emulated link (not a real ISP)
Later: home router, real ISP, gigabit

LINKS

GitHub: to be added
Live demo: to be added